pp_design: what runs, step by step

every call of the three main scripts, also the hidden ones inside `utils/`

Legend: `script step` `local function` (inside the same file) `utils function` `MATLAB function` **loop**

Which β is the good one?

β is **a choice you make**, not a result of the search. `pp_gains` only *computes* β^* from the poles ($\beta^* = a_0/(a_1\lambda_{slow})$, which in approach A is $1/(1+2\alpha\zeta^2)$). `pp_report` then shows the same poles with β^* and $\beta = 1$ side by side, compared with the PID, so you see which one meets the requirements. Rules from the theory:

- **steps** (A and B): β^* — the zero cancels the slowest real pole: A gives a clean 2nd-order step, B two real poles;
- $\beta = 0$ **is not an option**: the reference would enter only through the integrator, so the response would be driven by the slowest pole (about 2 s to settle with $\lambda_{min} = 2$ rad/s): much slower and not useful for our control. It was removed from the scripts;
- **trajectories** (square, circle): $\beta = 1$ — with $\beta < 1$ the joint lags by $(1-\beta)\frac{a_1}{a_0}\dot{r}$.

The `beta_mode` you set is the one the search uses to check the requirements and the one written to Simulink (`beta_xxxx`).

design_A.m

- design_A.m
  - 1. settings— start_from_pid, wn, zeta, alpha, beta_mode, do_search
  - 2. [mot, req] = pp_setup()
    - pp_setup— writes every requirement number into req
      - read_pid_tune— local function of pp_setup
        - run('.../pid_tune.m')— pid_tune runs parameters.m (k, tau) and pidtune (PID gains)
        - — copies k, tau, PID of both motors into a struct
      - make_motor— called twice: motor 3165 and 3166
        - ss— A, B, C, G of the motor; Ai, Bi with the integral state
        - eig— PID poles = eig(Ai - Bi*[Kp Kd -Ki])
  - 3. from_pid(mot(1).p_pid)— only if start_from_pid = true
    - from_pid— PID poles of motor 3165 → wn, zeta, alpha
  - 4. x = pp_search(@poles_A, [wn zeta alpha], beta_mode, mot, req)— only if do_search = true
    - poles_A(start)— start poles, the distance is measured from them
    - **repeat up to 4 times:** fminsearch— each try of new parameters calls:
      - search_cost— local function of pp_search
        - poles_A(exp(y))— parameters → 3 poles
        - **for each motor (3165, 3166):**
          - pp_gains(m, p)
            - acker(Ai, Bi, p)— k1, k2, ki
            - poly(p)— a1, a0 → beta*
            - acker(A', C', p_obs)— observer gain L
          - beta_number— 'star' → beta*, 'one' → 1
          - closed_loop(m, req, g, beta)
            - pp_controller— controller block: observer + feedback + integral
            - InputDelay, sumblk, connect— 3 ms delay, load d, closed loop
            - step— 10 deg step and load step → OS, ts, u_peak, IAE
            - margin— phase and gain margin
            - freqresp— chatter voltage
          - pp_check(r, p, req)— 6 violations v (v ≤ 0: met)
        - — keep the worst motor; J = distance + 1e4 * penalty
    - — stop when every v ≤ 0.001; return x = [wn zeta alpha]
    - p = poles_A([wn zeta alpha])— the final poles
  - 5a. pp_report(mot, req, p, beta_mode, 'Design A')
    - **for each motor:**
      - pp_gains— gains of this motor
      - closed_loop **x3**— PID (local pid_controller), beta*, beta = 1
      - pp_check **x3**— values and violations
      - fprintf— the table in the terminal ('*' = not met)
      - plot_steps— figure: poles, angle, voltage
  - 5b. pp_simulink(mot, req, p, beta_mode)
    - **for each motor:**
      - pp_gains— gains and beta of this motor
      - assignin('base', ...)— Kx_, Ki_, Kaw_, A_obs_, B_obs_, beta_, k_, tau_, PID gains
    - assignin— Vmax

design_B.m

Same structure as `design_A.m`; the differences are highlighted.

`design_B.m`

1. `settings` — start_from_pid, lambda, beta_mode ('star' or 'one'), do_search
2. `[mot, req] = pp_setup()` — same as design_A (read_pid_tune, make_motor)
3. `lambda = sort(abs(mot(1).p_pid))` — PID speeds = start
4. `pp_search( @poles_B , lambda, ...)`
   - same inside as design_A:
     - fminsearch → `search_cost` → `poles_B`
     - `pp_gains`, `closed_loop`, `pp_check` for both motors
5. `p = poles_B(lambda)` — three real poles

`poles_B` — sort the speeds, p = [-l1; -l2; -l3]

5a. `pp_report(mot, req, p, beta_mode, 'Design B')` — same as design_A

5b. `pp_simulink(mot, req, p, beta_mode)` — same as design_A

The only real difference is the pattern. In A the search moves $[\omega_n, \zeta, \alpha]$, so the pair moves as a pair; in B it moves $[\lambda_1, \lambda_2, \lambda_3]$, each real pole on its own. Everything below `search_cost` is identical.

check_design.m

No search: you give the poles, the script only checks them.

`check_design.m`

1. `settings` — mode, wn/zeta/alpha or p_hand, beta, motors_to_check
2. `[mot, req] = pp_setup()` — same as design_A
3. `build p` — 'A': from wn, zeta, alpha; 'poles': p = p_hand

`checks` — stable poles, complex ones in pairs (else `error`)

4. `pp_report(mot(motors_to_check), ...)` — only the chosen motors
   - `pp_gains`, `closed_loop` x3, `pp_check` x3, `plot_steps`
5. `pp_simulink(mot, req, p, beta)` — always both motors

Who calls whom (summary)

function	called by	calls
`pp_setup`	design_A, design_B, check_design, explore_poles	read_pid_tune (pid_tune.m), make_motor
`pp_search`	design_A, design_B	poles_A / poles_B, search_cost, pp_gains, closed_loop, pp_check
`pp_gains`	pp_search, pp_report, pp_simulink, explore_poles	acker, poly
`closed_loop`	pp_search, pp_report, explore_poles	pp_controller or pid_controller, connect, step, margin
`pp_check`	pp_search, pp_report, explore_poles	—
`pp_report`	design_A, design_B, check_design	pp_gains, closed_loop, pp_check, plot_steps
`pp_simulink`	design_A, design_B, check_design	pp_gains, assignin